AGENTIC SDLC · CLOUD MIGRATION REFERENCE MODEL

Autonomous AI Agents

and Their Role in the AI-Driven Software Development Lifecycle

Kubernetes applications · Datastores · Serverless & batch workloads · Legacy monolith modernization



Suhaib Khalid
Senior Technical Program Manager · PMP · PMI-ACP · AWS Solutions Architect

One program, four very different migration problems

Anonymized Q4 snapshot of an enterprise move to a hardened Kubernetes cloud platform (≈ 4 weeks into the quarter)

761
work items across 63 epics

131
delivered / verified (17%)

77
in active development

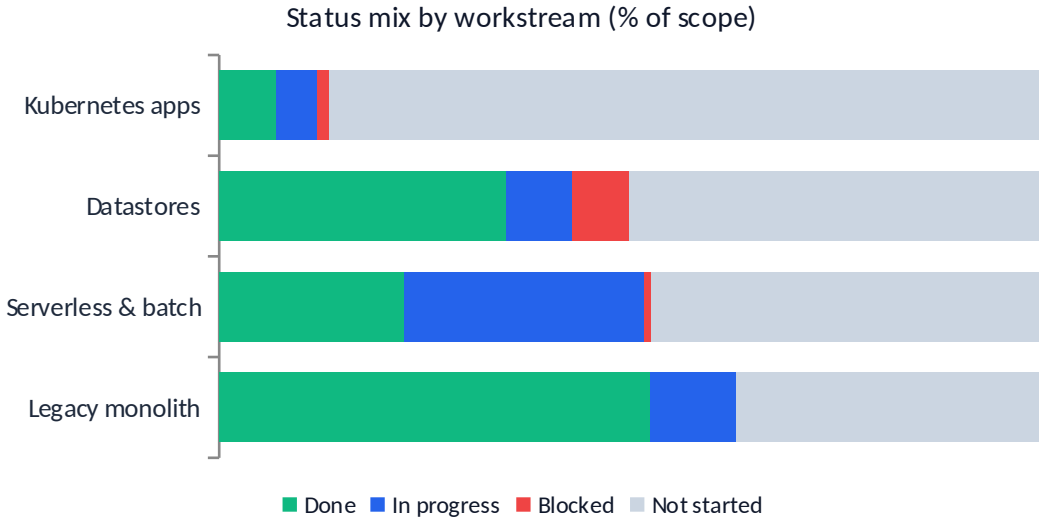
19
blocked — mostly on decisions & access, not code

Kubernetes applications
435 items · 9 epics

Datastores
174 items · 16 epics

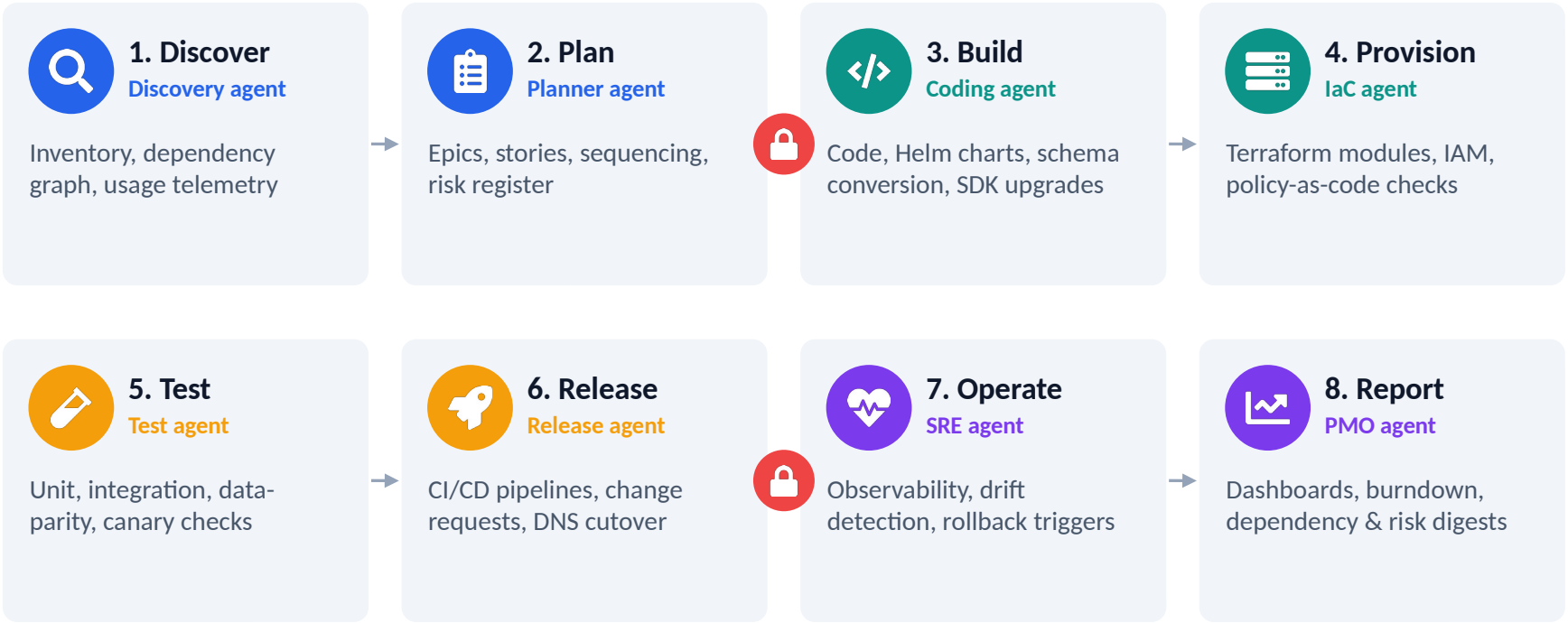
Serverless, batch & tooling
133 items · 29 epics

Legacy monolith
19 items · 9 epics



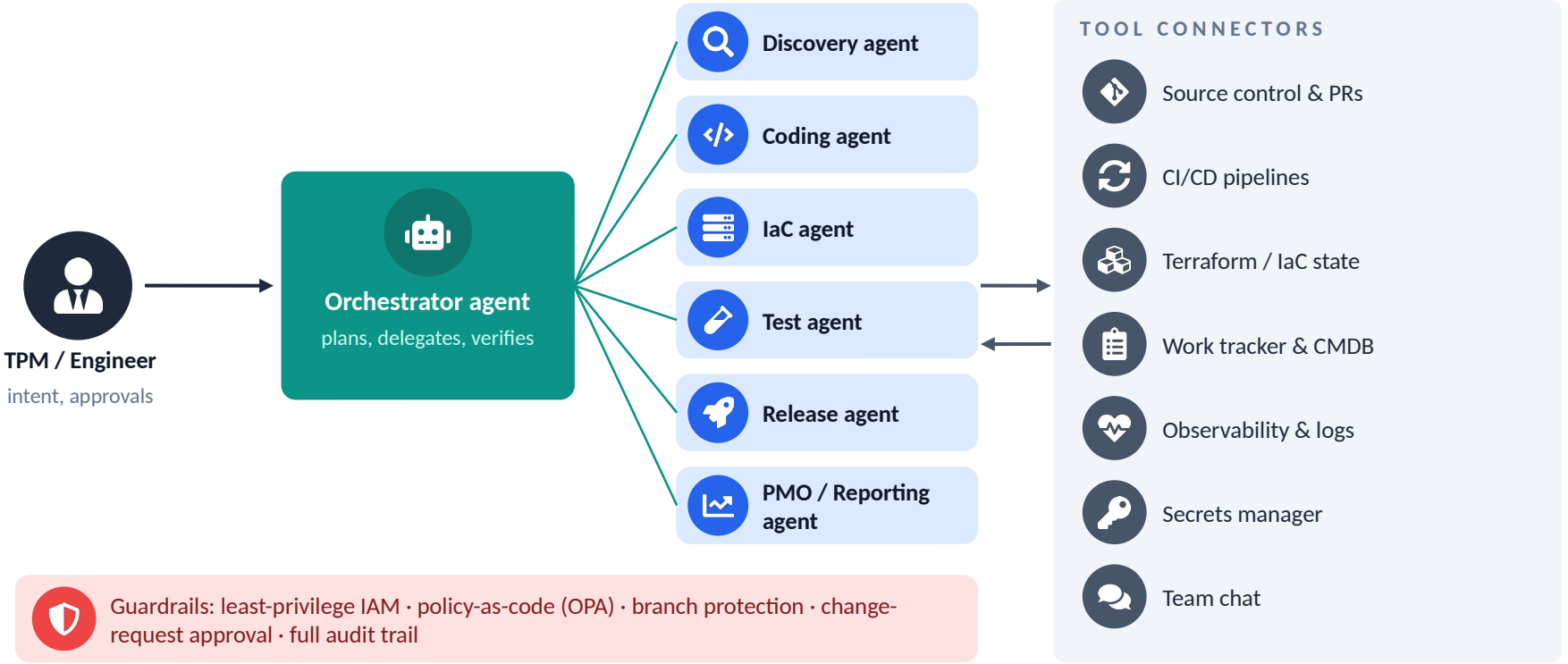
Agentic SDLC: specialist agents in every phase

Each phase is a loop of plan → act → verify, with a human approval gate before anything reaches production



Reference architecture: orchestrator, agents and tools

Agents reach enterprise systems through tool connectors (e.g., Model Context Protocol), never through standing credentials



WORKSTREAM 1

Kubernetes application migration



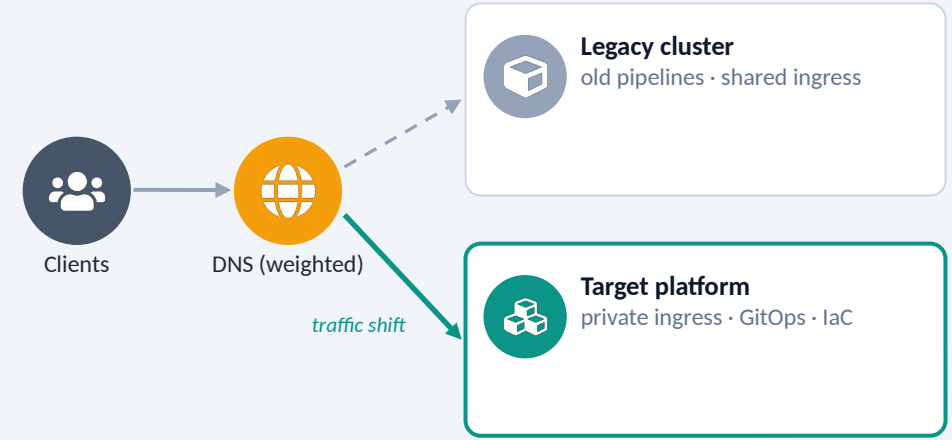
Containerized services move from a legacy cluster to a hardened target platform: pipelines, IaC, ingress and DNS cutover.

Kubernetes apps: a repeatable per-service migration factory

Same 7-step path for every service; agents do the repetitive build work and humans own the cutover decision



MIGRATION TOPOLOGY

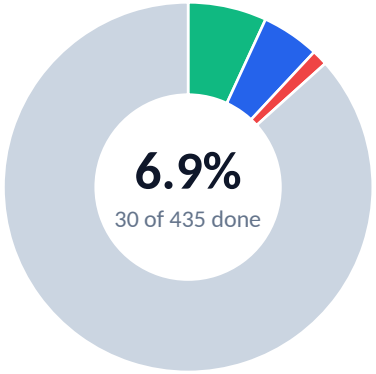


Where agents add leverage

- Generate pipeline workflows & Helm values per service
- Detect Terraform state drift before apply
- Watch canary latency; recommend rollback on gateway timeouts
- Draft change requests with evidence attached

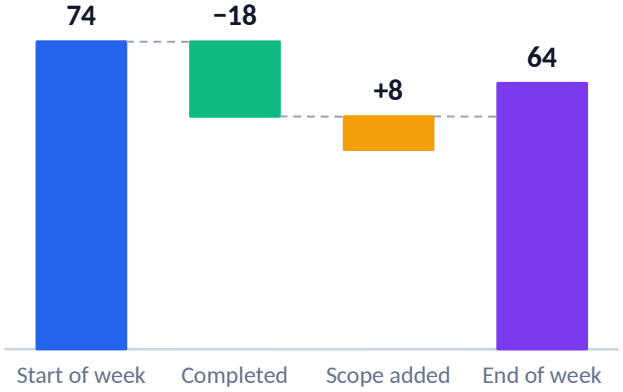
Kubernetes apps: early in the curve, velocity accelerating

435 items (77 deliverable stories/tasks + 349 sub-tasks); one week of status telemetry

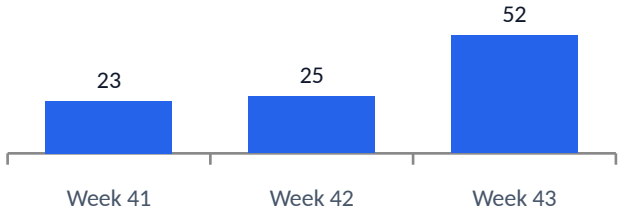


- Done 30
- In progress 22
- Blocked 6
- Not started 377

Weekly net-flux burndown (open items)



Week-over-week items touched



Risks surfaced in weekly telemetry

- **HIGH** — Legacy service retirement waits on a business decision about active users
- **MED** — Staging DNS cutover rolled back after private-ingress gateway timeouts
- **MED** — Message-broker sunset decision has held 2 services for ~300 days
- **LOW** — Change-approval lead time on cutover requests

WORKSTREAM 2

Migrating a variety of datastores



Relational, search, streaming, file and object stores: each needs its own pattern for conversion, replication, validation and cutover.

Datastores: one pipeline, five pattern variants

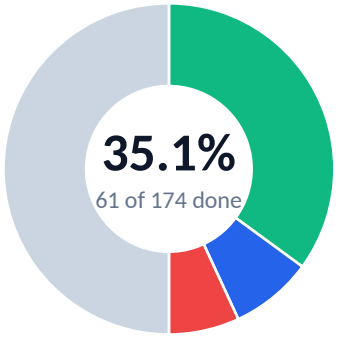
Agents generate the conversion and validation artifacts; the cutover window is always a human-scheduled event



Pattern	Example scope	What the agent produces
Engine conversion	Commercial SQL → PostgreSQL (2 APIs)	Schema & query rewrite, data-copy scripts, cutover runbook
Version upgrade / re-host	PostgreSQL upgrade on managed RDS	Upgrade plan, parameter diff, pre-checks
Search cluster	Elasticsearch auth hardening & v5 sunset	Client code for basic-auth, secrets wiring, index audit
Streaming / CDC	Kafka topic migration & cluster merge	Connector configs, topic mapping, observability
File & object storage	Network file system volume sync; buckets	Cross-account copy jobs, mount-target switch plan

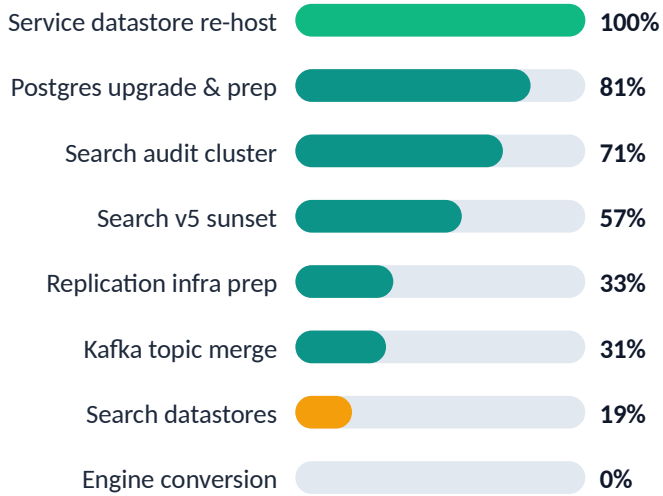
Datastores: furthest along, and where dependency chains bite

174 items across 16 epics; 12 blocked items are almost all chained prerequisites

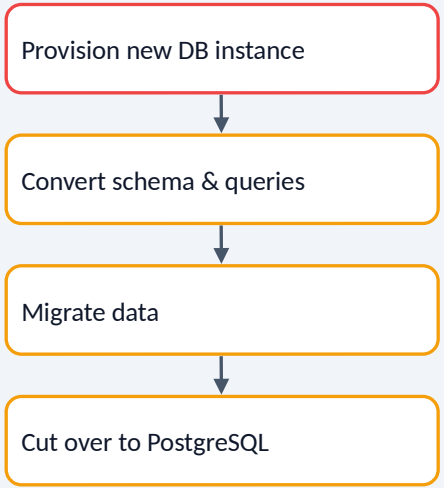


- Done 61
- In progress 14
- Blocked 12
- Not started 87

Completion by epic



Blocked dependency chain



One missing prerequisite holds 6 downstream items for 77 days

WORKSTREAM 3

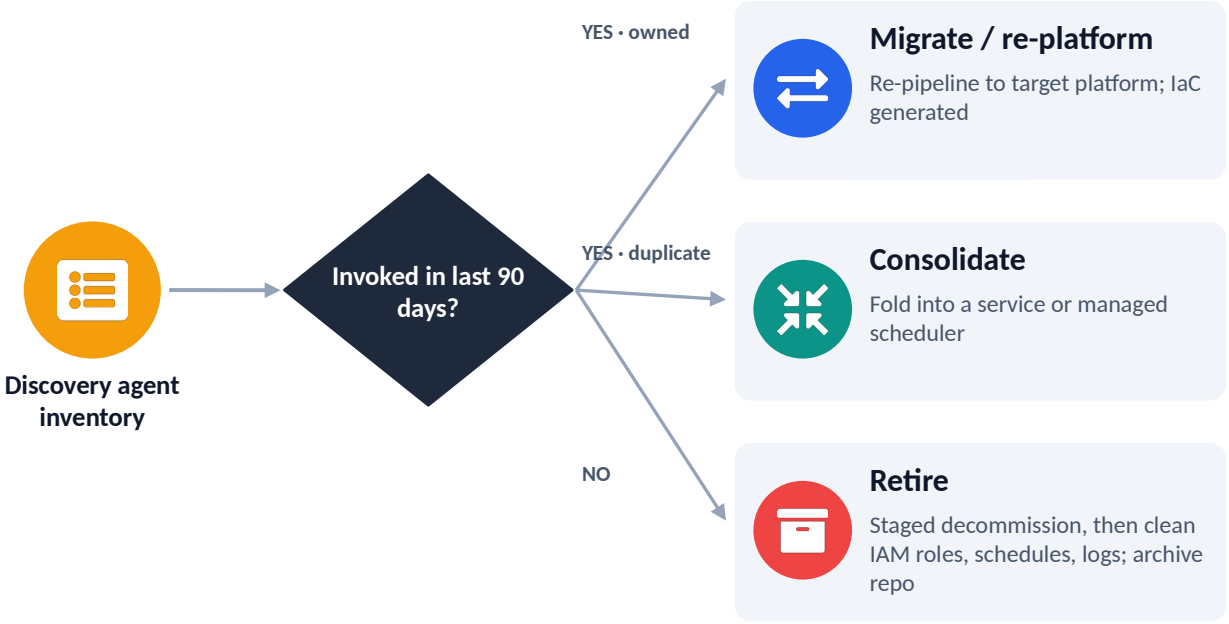
Serverless functions, batch jobs and platform tooling



The long tail: functions, scheduled jobs, build agents, deploy targets, snapshots and buckets. Migrate, modernize or retire.

Long tail: agents make migrate-or-retire evidence-based

Usage telemetry, ownership and dependencies decide each workload's path; humans confirm before anything is deleted



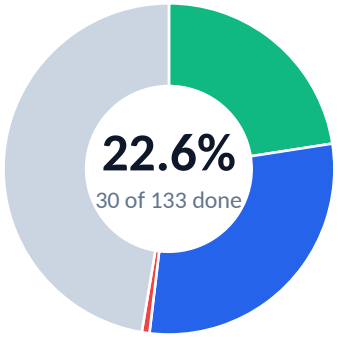
Workload types in scope

- Serverless functions
- Batch & cron jobs
- CI build agents
- Deployment targets
- Volume snapshots
- Object-storage buckets
- Legacy core networking

Guardrail: nothing is deleted until cross-team ownership is confirmed. One orphan-batch-job cleanup waited 60 days on exactly that audit.

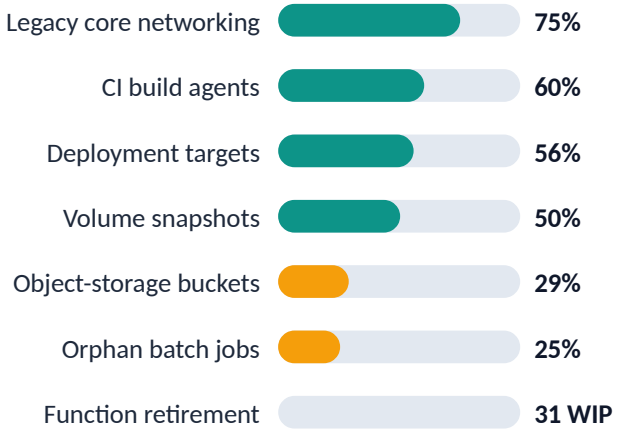
Long-tail workloads: highest throughput of the four streams

133 items across 29 epics; 13 items closed in one week, with 31 function retirements in flight

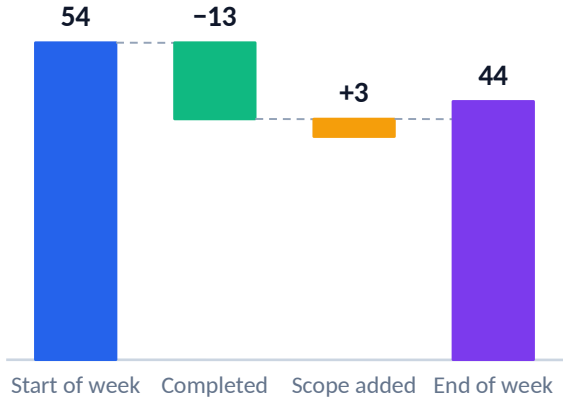


- Done 30
- In progress 39
- Blocked 1
- Not started 63

Completion by epic



Weekly net-flux burndown



12 functions retired in one week
across staging and production, with IAM roles, event schedules and log subscriptions cleaned up in the same change.

WORKSTREAM 4

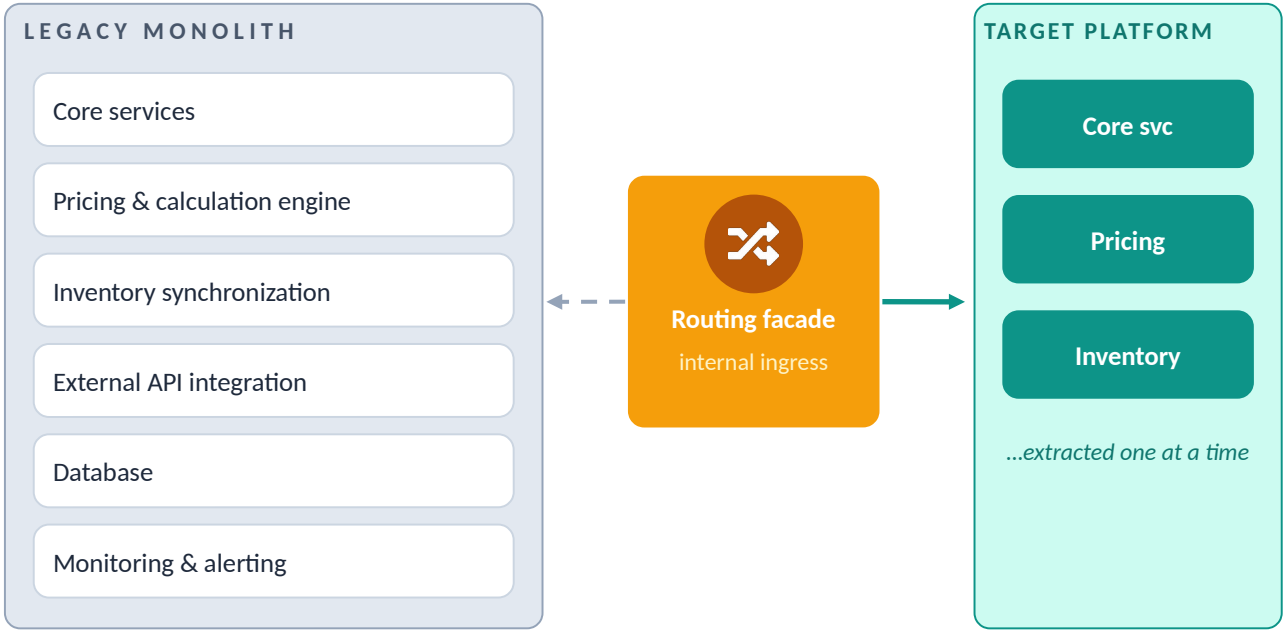
Modernizing a legacy monolith



A decades-old, semi-manual platform (config-managed servers, hand-run deploys) moves to the target platform, one component at a time.

Legacy monolith: agent-assisted strangler-fig extraction

A routing facade shifts traffic component by component, so the legacy system shrinks while the business keeps running



Agent roles

Code-comprehension agent
maps undocumented modules & call paths

Coding agent
cloud SDK upgrades across many repos

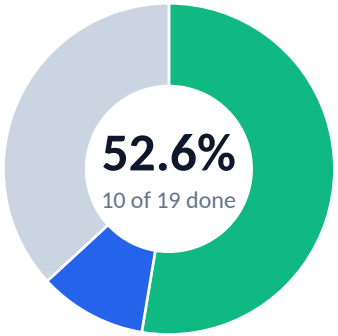
IaC agent
config-management playbooks → Terraform

Release agent
zero-downtime deploy patterns

Security win: the architecture review found the platform needed no internet-facing load balancer, so a perimeter risk was removed instead of migrated.

Legacy monolith: foundation done first, extraction follows

19 items in 9 epics; 90% of foundation stories and tasks complete, with zero blockers



- Done 10
- In progress 2
- Blocked 0
- Backlog epics 7

Phase 0 · Foundation — DONE

SDK upgrades in 5 teams, config-mgmt repo moved, zero-downtime deploys, QA infra as code

Phase 1 · Platform landing — IN PROGRESS

Terraform into a central environments repo; ingress hardening; policy exceptions

Phase 2 · Extraction — PLANNED

Core services, pricing, inventory sync, external APIs

Phase 3 · Data cutover — PLANNED

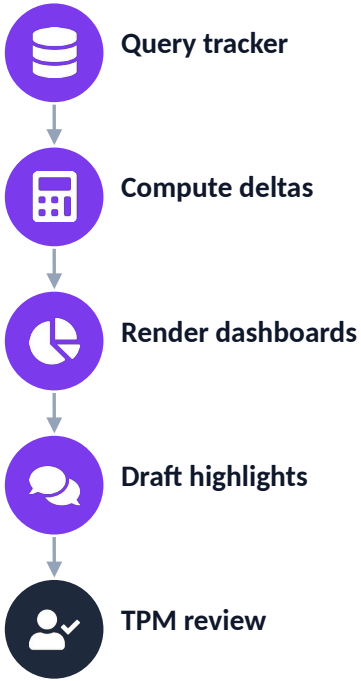
Database cutover & validation; monitoring re-platform

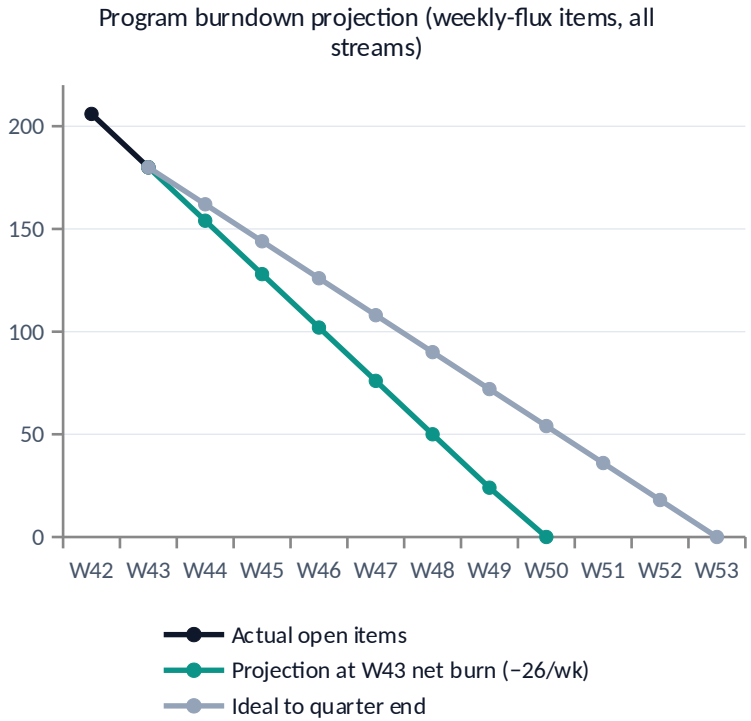
Phase 4 · Decommission — PLANNED

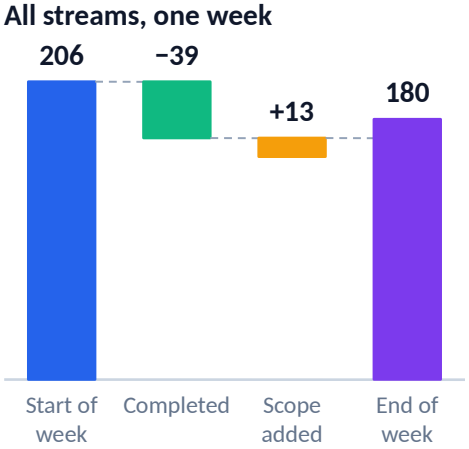
Retire legacy servers and pipelines

Governance: agents turn tracker data into weekly decisions

The pattern: a PMO agent queries the work tracker weekly; the TPM reviews, edits and publishes







Agent-generated artifacts
Status & weekly dashboards, burndowns, dependency maps, risk registers, chat-ready highlights

What agents changed, and what they did not

Code stopped being the bottleneck; decisions, access and approvals became the critical path

Agents accelerate

- Repetitive per-service build work (pipelines, Helm, Terraform)
- Cross-repo upgrades and schema conversion
- Inventory and usage analysis of the long tail
- Weekly status, burndown and risk reporting

Humans still own

- Scope approval and sequencing trade-offs
- Production change approval and cutover windows
- Business decisions (retire vs. keep for active users)
- Architecture calls, e.g. removing public ingress

New critical path

- Business decisions: up to ~300 days blocked
- Credentials & secrets provisioning
- Policy-as-code exceptions and IAM reviews
- Ownership audits before any deletion

Measuring an agentic SDLC

Track agent impact with the same delivery metrics engineering already trusts

Lead time for changes

Story start → production

Deployment frequency

Per service, per week

Change failure rate

Rollbacks per cutover

Time to restore

Detection → rollback

Agent acceptance rate

Agent PRs merged without rework

Blocked-days on critical path

Decision & access latency

DORA metrics + two agent-specific measures · Suhaib Khalid · linkedin.com/in/suhaibq